

源码可追溯技术文档

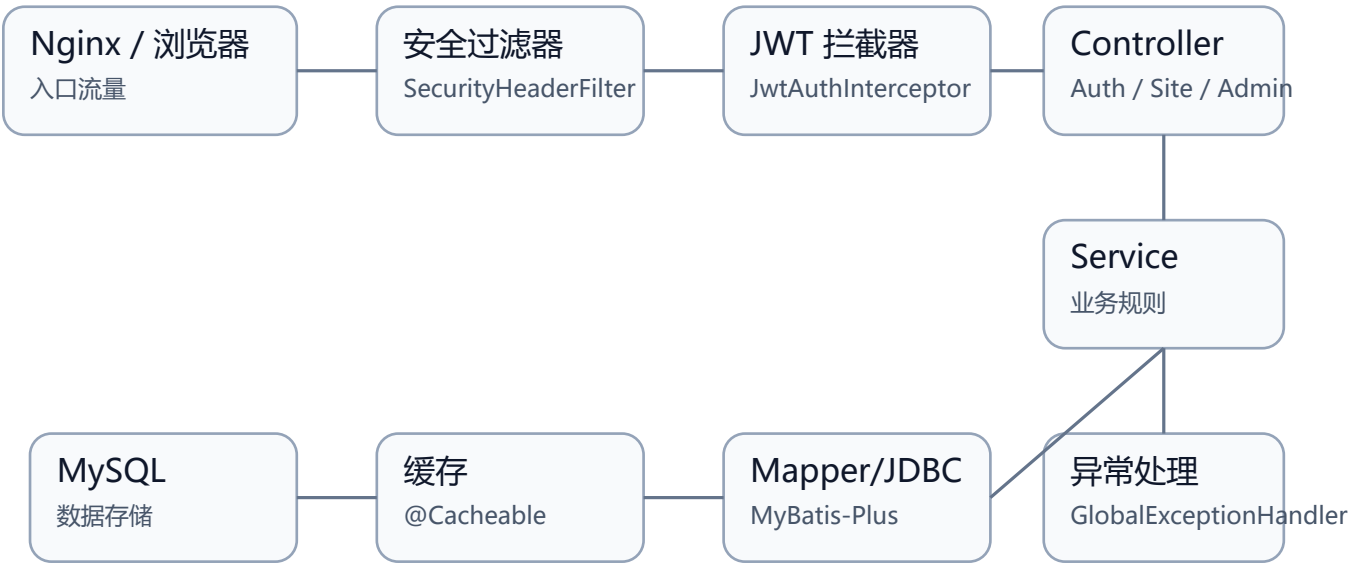
# 后端实现详解

来源文件: backend.md

生成方式: Python + ReportLab (当前环境下的最小可控方案)

特点: 可点击目录、源码锚点清晰、图表内嵌、代码块可读

## 请求生命周期图



图示说明：请求从 Nginx 进入后，依次经过安全头过滤、JWT 拦截、控制器与服务层，再进入 Mapper/JDBC 与缓存。

# 目录

---

- 目录
- 1. 项目整体架构
  - 1.1 架构总览
  - 1.2 分层结构
  - 1.3 数据流向
- 2. 模块划分与职责说明
  - 2.1 控制器层
  - 2.2 服务层
  - 2.3 基础设施层
  - 2.4 持久化层
- 3. 关键依赖与版本清单
- 4. 功能逐点拆解
  - 4.1 应用启动与基础设施装配
  - 4.2 WebMvc 配置：CORS、拦截器、媒体映射
  - 4.3 JWT 生成与后台登录
  - 4.4 登录限速与密码升级
  - 4.5 JWT 鉴权拦截与线程上下文
  - 4.6 前台聚合接口与缓存
  - 4.7 留言提交与 XSS 清洗
  - 4.8 后台内容管理与缓存失效
  - 4.9 访问日志、仪表盘与事务边界
  - 4.10 媒体上传、本地/OSS 存储与代理访问
  - 4.11 安全配置绑定与跨域白名单
- 5. 数据流与状态管理
  - 5.1 请求生命周期
  - 5.2 缓存边界
  - 5.3 线程上下文边界
  - 5.4 事务边界
  - 5.5 异常处理流程
  - 5.6 配置绑定边界
- 6. 安全与性能
  - 6.1 鉴权与密码安全
  - 6.2 内容安全
  - 6.3 XSS 与输入边界
  - 6.4 上传安全
  - 6.5 来源识别与 HTTPS 判断
  - 6.6 路径穿越与对象存储安全

- 6.7 当前未发现的安全/稳定性能力
- 7. 测试与质量保障
  - 5.1 请求生命周期
  - 5.2 缓存边界
  - 5.3 事务边界
  - 5.4 异常处理流程
- 6. 安全与性能
  - 6.1 鉴权与密码安全
  - 6.2 内容安全
  - 6.3 XSS 与输入边界
  - 6.4 上传安全
  - 6.5 路径穿越与对象存储安全
  - 6.6 当前未发现的安全/稳定性能力
- 7. 测试与质量保障
  - 7.1 测试现状
  - 7.2 自动化流水线
  - 7.3 代码规范
- 8. 部署与运维
  - 8.1 环境配置
  - 8.2 打包与运行
  - 8.3 反向代理
  - 8.4 存储模式
  - 8.5 当前未发现的运维能力
- 9. 现状结论与补充建议
  - 9.1 已具备能力
  - 9.2 当前未发现或未实现能力
  - 9.3 读者复现建议

# 目录

- [1. 项目整体架构](#1-项目整体架构)
- [2. 模块划分与职责说明](#2-模块划分与职责说明)
- [3. 关键依赖与版本清单](#3-关键依赖与版本清单)
- [4. 功能逐点拆解](#4-功能逐点拆解)
- [5. 数据流与状态管理](#5-数据流与状态管理)
- [6. 安全与性能](#6-安全与性能)
- [7. 测试与质量保障](#7-测试与质量保障)
- [8. 部署与运维](#8-部署与运维)
- [9. 现状结论与补充建议](#9-现状结论与补充建议)

## 1. 项目整体架构

### 1.1 架构总览

后端位于 backend/, 使用 Spring Boot 3.3.1 构建 REST API, 数据访问采用 MyBatis-Plus, 认证采用 JWT, 数据库为 MySQL, 并支持本地存储与阿里云 OSS 两种媒体资源存储模式。

应用入口位于 backend/src/main/java/com/dyx/blog/BlogApplication.java:11-23:

- @SpringBootApplication 启动 Spring 容器。
- @EnableCaching 开启缓存能力。
- @MapperScan("com.dyx.blog.mapper") 自动扫描 Mapper。

整体生命周期图见 ../assets/backend-lifecycle.mmd。

### 1.2 分层结构

后端基本遵循以下调用链:

Nginx -> Filter -> Interceptor -> Controller -> Service -> Mapper/JdbcTemplate -> MySQL -> Result

异常统一由 GlobalExceptionHandler 收敛, 见 backend/src/main/java/com/dyx/blog/common/exception/GlobalExceptionHandler.java:20-128。

### 1.3 数据流向

- 公开站点: /api/site/\*\*
- 登录入口: /api/auth/login
- 后台管理: /api/dyx-manager/\*\*
- 媒体资源访问: /media/\*\* 或 /api/dyx-manager/media/content

## 2. 模块划分与职责说明

### 2.1 控制器层

- controller/auth/AuthController.java: 后台登录。

- controller/publics/SiteController.java: 公开站点接口。
- controller/admin/AdminController.java: 后台管理接口。

2.2 服务层

- service/impl/AuthServiceImpl.java: 登录认证、密码校验、JWT 签发。
- service/impl/SiteServiceImpl.java: 公开站点聚合查询、留言提交、访问统计。
- service/impl/AdminServiceImpl.java: 后台 CRUD、统计查询、系统配置管理。
- service/impl/MediaServiceImpl.java: 媒体上传、导入、代理访问、引用校验。

2.3 基础设施层

- common/interceptor/JwtAuthInterceptor.java: 后台接口 JWT 鉴权。
- common/filter/SecurityHeaderFilter.java: 统一安全响应头。
- common/util/JwtUtil.java: JWT 生成与解析。
- common/util/LoginAttemptLimiter.java: 登录频率限制。
- common/util/XssUtil.java: 文本/HTML 清洗。
- common/util/ClientIpUtil.java: 客户端 IP 与 HTTPS 来源识别。
- common/context/UserContext.java: 当前请求用户上下文。
- config/DyxSecurityProperties.java: 跨域与加密相关配置绑定。
- config/FileProperties.java: 文件上传目录与存储模式绑定。
- storage/LocalMediaStorage.java: 本地文件存储。
- storage/OssMediaStorage.java: 阿里云 OSS 存储。

2.4 持久化层

Mapper 接口集中在 backend/src/main/java/com/dyx/blog/mapper/，覆盖：

- PostMapper
- MomentMapper
- ProjectMapper
- WorkMapper
- ProfileMapper
- UserMapper
- MediaMapper
- FootprintMapper
- GuestbookMessageMapper
- SystemConfigMapper
- SiteVisitLogMapper

3. 关键依赖与版本清单

| 依赖                         | 版本    | 作用   |
|----------------------------|-------|------|
| spring-boot-starter-parent | 3.3.1 | 基础框架 |

| 依赖                                | 版本          | 作用            |
|-----------------------------------|-------------|---------------|
| java                              | 17          | 运行时版本         |
| mybatis-plus-spring-boot3-starter | 3.5.7       | ORM/Mapper 增强 |
| jjwt                              | 0.12.6      | JWT 生成与校验     |
| mysql-connector-j                 | runtime     | MySQL 驱动      |
| spring-security-crypto            | 跟随 Boot     | BCrypt 密码加密   |
| jsoup                             | 1.18.1      | HTML 清洗       |
| alibabacloud-oss-v2               | 0.3.1       | OSS 存储        |
| httpclient5/httpcore5             | 5.5 / 5.3.4 | HTTP 访问能力     |

来源：backend/pom.xml:5-127

## 4. 功能逐点拆解

### 4.1 应用启动与基础设施装配

场景

后端服务启动时需要注册 Spring 容器、Mapper 扫描与缓存能力。

需求

统一装配基础设施，避免额外 XML 或手工注册。

实现方案

在 BlogApplication.java:11-23 使用注解完成启动配置。

选型理由

Spring Boot 注解驱动方式简洁，适合当前中小型博客后端。

核心源码定位

- backend/src/main/java/com/dyx/blog/BlogApplication.java:11-23

```
java
@SpringBootApplication
@EnableCaching
@MapperScan("com.dyx.blog.mapper")
public class BlogApplication {
    public static void main(String[] args) {
        SpringApplication.run(BlogApplication.class, args);
    }
}
```

替代方案对比

- 替代方案：XML 扫描或手工注册 Mapper。
- 弃用原因：样板代码更多，不符合 Spring Boot 现代配置方式。

## 4.2 WebMvc 配置：CORS、拦截器、媒体映射

### 场景

项目既要支持前端跨域开发，又要保护后台接口，同时暴露本地上传媒体。

### 需求

- 配置跨域白名单。
- 对后台管理 API 开启 JWT 拦截。
- 为 /media/\*\* 建立静态文件访问映射。

### 实现方案

backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java:21-74 完成三项配置：

1. addCorsMappings() 读取 dyx.security.cors-allowed-origins。
1. addInterceptors() 仅拦截 /api/dyx-manager/\*\*。
1. addResourceHandlers() 将 file.upload-path 暴露为静态资源。

### 选型理由

使用 Spring MVC 原生扩展点即可满足要求，无需引入完整 Spring Security 过滤链接管所有行为。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java:42-50
- backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java:57-62
- backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java:69-73

java

```
registry.addInterceptor(dyxJwtAuthInterceptor)
    .addPathPatterns("/api/dyx-manager/**")
    .excludePathPatterns("/api/auth/login", "/api/dyx-manager/media/content");
```

### 替代方案对比

- 替代方案：Spring Security 全量接管认证与资源规则。
- 弃用原因：功能更全，但复杂度更高；当前项目只需 JWT + 路径级保护，MVC 拦截器实现成本更低。

## 4.3 JWT 生成与后台登录

### 场景

管理员登录后需要生成一个包含用户标识与角色的 token，用于后续后台接口访问。

### 需求

- 校验用户名/密码。
- 判断账号启用状态与后台角色。
- 返回 token 与用户信息。

### 实现方案

1. AuthController.java:33-36 接收登录请求。

1. AuthServiceImpl.java:40-84 负责:

- 登录限速
- 查询用户
- 验证密码
- 检查启用状态
- 检查角色是否为管理员
- 生成 JWT

1. JwtUtil.java:32-41 使用 userId 和 role 生成 token。

#### 选型理由

JWT 适合当前前后端分离后台场景，不依赖服务端 Session 存储，部署简单。

#### 核心源码定位

- backend/src/main/java/com/dyx/blog/controller/auth/AuthController.java:33-36
- backend/src/main/java/com/dyx/blog/service/impl/AuthServiceImpl.java:40-84
- backend/src/main/java/com/dyx/blog/common/util/JwtUtil.java:32-41

java

```
String token = dyxJwtUtil.generateToken(user.getId(), user.getRole());
UserVo userVo = new UserVo();
userVo.setId(user.getId());
userVo.setUsername(user.getUsername());
userVo.setDisplayName(user.getDisplayName());
userVo.setRole(user.getRole());
return new LoginResponse(token, userVo);
```

#### 替代方案对比

- 替代方案: 服务端 Session。
- 弃用原因: 需要额外会话存储与粘性会话考虑，不如 JWT 轻便。
- 替代方案: OAuth2。
- 弃用原因: 当前是单站点后台管理，不需要引入外部授权协议复杂度。

## 4.4 登录限速与密码升级

#### 场景

后台登录既要防爆破，也要兼容历史明文密码并逐步升级为 BCrypt。

#### 需求

- 连续失败过多时限制登录。
- 旧密码匹配成功后自动升级为 BCrypt。

#### 实现方案

- LoginAttemptLimiter.java:15-76 基于内存 ConcurrentHashMap 对用户名与 IP 分别限速。
- AuthServiceImpl.java:86-105 中的 matchesPassword():
  - 若已是 BCrypt，则走 passwordEncoder.matches()。
  - 若还是旧明文密码，匹配成功后立即回写 BCrypt 密码。

#### 选型理由



这种“登录时迁移”策略避免了批量重置密码，对已有数据兼容性好。

#### 核心源码定位

- backend/src/main/java/com/dyx/blog/common/util/LoginAttemptLimiter.java:15-76
- backend/src/main/java/com/dyx/blog/service/impl/AuthServiceImpl.java:86-105

java

```
if (looksLikeBcrypt(storedPassword)) {
    return passwordEncoder.matches(rawPassword, storedPassword);
}
boolean matches = storedPassword.equals(rawPassword);
if (matches) {
    user.setPassword(passwordEncoder.encode(rawPassword));
    dyxUserMapper.updateById(user);
}
```

#### 替代方案对比

- 替代方案：强制所有用户统一重置密码。
- 弃用原因：安全上更彻底，但运维与用户体验成本更高。

## 4.5 JWT 鉴权拦截与线程上下文

#### 场景

后台接口在进入控制器前需要完成 token 校验，并把用户信息放入当前线程上下文。

#### 需求

- 校验 Authorization 头。
- 解析 token。
- 获取 userId 与 role。
- 请求结束后清理上下文。
- 让服务层可直接判断当前用户是否为管理员。

#### 实现方案

backend/src/main/java/com/dyx/blog/common/interceptor/JwtAuthInterceptor.java:33-73  
在 preHandle() 中解析 Bearer Token，并写入 UserContext；在 afterCompletion()  
中清理线程变量。

backend/src/main/java/com/dyx/blog/common/context/UserContext.java:8-65 通过两个  
ThreadLocal 分别保存 userId 与 role，并提供 isAdmin() 这样的便捷判断方法，供服务层直接复用。

#### 选型理由

对当前项目来说，MVC 拦截器 + ThreadLocal 上下文足以支撑后台鉴权和角色判断，接入点清晰；同时 UserContext 让 Controller 与 Service 不需要层层传递用户信息。

#### 核心源码定位

- backend/src/main/java/com/dyx/blog/common/interceptor/JwtAuthInterceptor.java:34-59
- backend/src/main/java/com/dyx/blog/common/interceptor/JwtAuthInterceptor.java:70-73
- backend/src/main/java/com/dyx/blog/common/context/UserContext.java:8-65

java

```
Claims claims = dyxJwtUtil.parseClaims(token);
Long userId = Long.valueOf(claims.getSubject());
String role = claims.get("role", String.class);
UserContext.setUserId(userId);
UserContext.setUserRole(role);
```

java

```
public static boolean isAdmin() {
    return "ADMIN".equalsIgnoreCase(getUserRole());
}

public static void clear() {
    USER_ID_HOLDER.remove();
    USER_ROLE_HOLDER.remove();
}
```

### 替代方案对比

- 替代方案：在每个 Controller 手工解析 token。
- 弃用原因：重复逻辑过多，且容易漏掉权限校验。
- 替代方案：把当前用户对象作为参数层层传给 Service。
- 弃用原因：当前后端层级较浅、请求内上下文更自然，ThreadLocal 成本更低。

## 4.6 前台聚合接口与缓存

### 场景

公开站点首页、博客、动态、项目、荣誉、足迹等数据读取频繁，适合缓存优化。

### 需求

- 首页需要聚合多类内容。
- 常用公开数据应具备缓存能力。
- 文章详情需要增加阅读量。

### 实现方案

SiteServiceImpl.java:77-357 提供以下能力：

- getHomeData() 聚合首页内容。
- getProfile()、listPosts()、listMoments()、listProjects()、listWorks()、listHonors()、listFootprints() 使用 @Cacheable。
- getPostDetail() 使用 JdbcTemplate 原子递增浏览量。

### 选型理由

- 聚合接口降低前端请求次数。
- 常用公开数据缓存可降低数据库压力。
- 对浏览量自增场景，直接 SQL update 比先查后改更直接。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java:77-88
- backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java:95-107

- backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java:160-174
- backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java:182-190

```
java
```

```
jdbcTemplate.update("UPDATE dyx_post SET view_count = COALESCE(view_count, 0) + 1 WHERE id = ?", id);
post.setViewCount((post.getViewCount() == null ? 0 : post.getViewCount()) + 1);
```

### 替代方案对比

- 替代方案：前端分别请求多个接口。
- 弃用原因：首页链路更长。
- 替代方案：JPA 实体更新浏览量。
- 弃用原因：为简单计数引入实体状态管理开销，不如 JDBC 直接。

## 4.7 留言提交与 XSS 清洗

### 场景

公开留言板是典型的用户输入边界，必须进行内容校验与过滤。

### 需求

- 限制空内容与最大长度。
- 存储访客 IP。
- 移除恶意 HTML。
- 在反向代理场景下尽量拿到真实来源地址。

### 实现方案

SiteServiceImpl.java:133-153 在保存留言前调用 XssUtil.cleanText(); 同时保存 ClientIpUtil.resolveClientIp(request) 的结果。

backend/src/main/java/com/dyx/blog/common/util/ClientIpUtil.java:12-39 会优先读取 request.getRemoteAddr(), 当来源被识别为受信代理时, 再尝试从 X-Forwarded-For 与 X-Real-IP 中提取真实客户端地址, 并统一做截断与本地回环归一化。

XssUtil.java:19-44 提供两类过滤:

- cleanText(): 纯文本字段, 移除所有 HTML。
- cleanHtml(): 富文本字段, 保留白名单标签。

### 选型理由

用 Jsoup 白名单比完全信任前端更可靠, 也比手写正则过滤更安全; 同时把 IP 解析逻辑集中到工具类, 能避免控制器和服务层反复处理代理头细节。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java:133-153
- backend/src/main/java/com/dyx/blog/common/util/XssUtil.java:19-44
- backend/src/main/java/com/dyx/blog/common/util/ClientIpUtil.java:12-39

```
java
```

```
nextMessage.setContent(XssUtil.cleanText(content));
nextMessage.setIpAddress(ClientIpUtil.resolveClientIp(request));
```

java

```
if (isTrustedProxy(remoteAddr)) {
    String forwardedIp = extractForwardedClientIp(request.getHeader("X-Forwarded-For"));
    if (!isBlank(forwardedIp)) {
        return truncate(forwardedIp, 45);
    }
}
```

### 替代方案对比

- 替代方案：只在前端转义。
- 弃用原因：服务端才是最终信任边界；不能假设所有请求都来自受控前端。
- 替代方案：业务代码直接读取请求头中的 IP。
- 弃用原因：代理信任、IPv6 兼容和长度限制逻辑容易分散且不一致。

## 4.8 后台内容管理与缓存失效

### 场景

后台维护文章、动态、项目、作品、荣誉等内容，修改后前台缓存必须及时失效。

### 需求

- 支持 CRUD。
- 保存时完成必要的字段标准化与富文本清洗。
- 修改后清理公开缓存。

### 实现方案

AdminServiceImpl.java 中对各内容类型采用统一模式：

- listXxx() 查询后台数据。
- saveXxx() 负责新增/更新。
- deleteXxx() 负责删除。
- 公开侧缓存通过 @CacheEvict 驱逐。

例如文章保存见 AdminServiceImpl.java:231-267，会清洗 HTML 内容并驱逐 site:posts 缓存。

### 选型理由

服务层统一收口业务规则，避免 Controller 直接拼装数据库操作。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:231-267
- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:296-321
- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:343-365
- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:392-415

### 替代方案对比

- 替代方案：不做缓存驱逐，依赖缓存自动过期。
- 弃用原因：后台保存后前台可能长时间看到旧数据，不利于管理体验。

## 4.9 访问日志、仪表盘与事务边界

### 场景

后台需要展示访问趋势、设备分布、页面热度，并支持批量删除访问日志。

### 需求

- 提供统计摘要。
- 分页查询访问日志。
- 超级管理员可批量删除。

### 实现方案

- 仪表盘汇总: AdminServiceImpl.java:86-99
- 分页查询: AdminServiceImpl.java:107-112
- 批量删除: AdminServiceImpl.java:132-142 使用 @Transactional

### 选型理由

批量删除属于多记录写操作，放在事务内可以保证失败时回滚一致性。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:86-99
- backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java:132-142

java

```
@Override
@Transactional
public void deleteVisitLogs(List<Long> ids) {
    if (!UserContext.isAdmin()) {
        throw new BusinessException(403, " ");
    }
    dyxSiteVisitLogMapper.deleteBatchIds(ids);
}
```

### 替代方案对比

- 替代方案: Controller 直接循环删除。
- 弃用原因: 职责下沉不合理，且缺少事务边界。

## 4.10 媒体上传、本地/OSS 存储与代理访问

### 场景

后台需要管理图片、PDF、视频等媒体资源，既支持本地文件，也支持 OSS。

### 需求

- 上传时校验大小、扩展名、MIME 类型与图片内容。
- 支持切换本地/OSS 存储。
- 删除前检查是否仍被其他业务引用。
- 支持受控代理访问媒体内容。

- 存储目录与访问前缀要能通过配置绑定统一控制。

### 实现方案

MediaServiceImpl.java:62-318 是媒体业务核心:

- 上传校验: validateUpload()。
- 本地文件导入: importExistingFiles()。
- 删除时通过 resolveReferenceModule() 判断是否仍被引用。
- 本地走 LocalMediaStorage, 远程走 OssMediaStorage。

backend/src/main/java/com/dyx/blog/config/FileProperties.java:10-23 通过 @ConfigurationProperties(prefix = "file") 绑定 storageType、uploadPath 与 accessPrefix, 让存储层与 MVC 资源映射共享同一份配置来源。

LocalMediaStorage.java:59-89 使用路径规范化和目录边界检查防止路径穿越。

OssMediaStorage.java:176-204 通过环境变量读取 OSS 凭证, 并校验系统配置完整性。

### 选型理由

抽象 MediaStorage 接口后, 本地与 OSS 的差异被收敛到存储层, 业务层无需关心底层介质; 再配合 FileProperties 集中绑定配置, 可以避免上传路径在多个类中写死。

### 核心源码定位

- backend/src/main/java/com/dyx/blog/service/impl/MediaServiceImpl.java:224-243
- backend/src/main/java/com/dyx/blog/service/impl/MediaServiceImpl.java:206-221
- backend/src/main/java/com/dyx/blog/config/FileProperties.java:10-23
- backend/src/main/java/com/dyx/blog/storage/LocalMediaStorage.java:69-89
- backend/src/main/java/com/dyx/blog/storage/OssMediaStorage.java:176-204

java

```
@ConfigurationProperties(prefix = "file")
public class FileProperties {
    private String storageType = "local";
    private String uploadPath = "uploads/";
    private String accessPrefix;
}
```

java

```
if (!resolvedPath.startsWith(uploadDirectory)) {
    throw new BusinessException("        ");
}
```

### 替代方案对比

- 替代方案: 所有文件都只放本地磁盘。
- 弃用原因: 简单, 但不适合未来扩展到对象存储。
- 替代方案: 业务层直接写 OSS SDK。
- 弃用原因: 会把存储细节泄漏到业务代码, 难以切换存储模式。
- 替代方案: 把上传目录硬编码在存储类中。
- 弃用原因: 部署环境切换时需要改代码, 不如配置绑定灵活。

## 4.11 安全配置绑定与跨域白名单

## 场景

后端既要支持前端开发环境跨域访问，又要把安全相关参数集中管理，避免散落在多个类中。

## 需求

- 统一读取跨域来源配置。
- 统一读取加密密钥相关配置。
- 让 WebMvc 配置和业务组件共享同一份配置源。

## 实现方案

backend/src/main/java/com/dyx/blog/config/DyxSecurityProperties.java:13-27 通过 @ConfigurationProperties(prefix = "dyx.security") 绑定 encryptKey 与 corsAllowedOrigins。

WebMvcConfig.java:42-50 在 addCorsMappings() 中直接使用该配置对象，把允许来源模式下发到 Spring MVC 的 CORS 配置。

## 选型理由

把安全相关配置聚合为类型安全的配置类，比在各处用 @Value 零散读取更清晰，也更容易被文档和运维脚本追踪。

## 核心源码定位

- backend/src/main/java/com/dyx/blog/config/DyxSecurityProperties.java:13-27
- backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java:42-50

java

```
@ConfigurationProperties(prefix = "dyx.security")
public class DyxSecurityProperties {
    private String encryptKey = "dyx-blog-default-key-32chars!!!";
    private List<String> corsAllowedOrigins = new ArrayList<>();
}
```

## 替代方案对比

- 替代方案：在每个配置点单独使用 @Value。
- 弃用原因：属性名分散、默认值分散，不利于维护。
- 替代方案：把 CORS 白名单写死在 WebMvcConfig。
- 弃用原因：不同环境切换成本更高，也不利于生产部署。

## 5. 数据流与状态管理

### 5.1 请求生命周期

典型后台请求流程：

1. Nginx 反向代理到 /api/dyx-manager/\*\*
1. SecurityHeaderFilter 写入安全头
1. JwtAuthInterceptor 校验 token 并写入 UserContext
1. Controller 调用 Service

## 1. Service 调用 Mapper / JdbcTemplate

1. 出错时由 `GlobalExceptionHandler` 统一转换响应
1. 请求结束后 `UserContext.clear()` 清理线程上下文

其中 `ClientIpUtil.java:30-39` 还负责根据 `request.isSecure()` 与 `X-Forwarded-Proto` 判断请求是否应视为 HTTPS，请求安全策略并不完全依赖应用服务器直连信息。

## 5.2 缓存边界

- 开启点: `BlogApplication.java:11-13`
- 读取缓存: 如 `SiteServiceImpl.java:95-107, 160-174, 199-287`
- 驱逐缓存: 如 `AdminServiceImpl.java:167-168, 232-233, 275-276, 297-320`

缓存明显偏向“公开站点读取多、后台修改少”的博客业务模型。

## 5.3 线程上下文边界

`backend/src/main/java/com/dyx/blog/common/context/UserContext.java:8-65`  
显式把用户身份限定在“单次请求线程”内:

- 进入拦截器后写入 `userId` 与 `role`
- Service 层可通过 `UserContext.isAdmin()` 做权限分支
- 请求结束必须 `clear()`，避免线程池复用导致脏数据串请求

这体现出该项目的状态管理重点不是复杂会话存储，而是请求级最小上下文。

## 5.4 事务边界

当前已明确看到 `@Transactional` 用于批量删除访问日志 (`AdminServiceImpl.java:132-142`)。说明项目没有滥用事务，而是针对多步写操作按需开启。

## 5.5 异常处理流程

`GlobalExceptionHandler.java:30-128` 将异常分为:

- `BusinessException`
- 参数校验异常
- 请求方法异常
- 通用兜底异常

返回统一 `Result.failure(...)` 结构，便于前端 `Axios` 拦截器统一处理。

## 5.6 配置绑定边界

后端配置明显采用“配置类收口”的方式:

- `DyxSecurityProperties` 负责安全与跨域属性
- `FileProperties` 负责上传与存储属性

这使 `WebMvcConfig`、存储层和安全工具不需要自己解析原始配置字符串。

# 6. 安全与性能

## 6.1 鉴权与密码安全

- JWT 签发与校验: `JwtUtil.java:32-55`



- BCrypt 密码校验与升级: AuthServiceImpl.java:86-105
- 登录限速: LoginAttemptLimiter.java:15-76

## 6.2 内容安全

SecurityHeaderFilter.java:31-46 统一设置:

- X-Frame-Options
- X-Content-Type-Options
- X-XSS-Protection
- Content-Security-Policy
- Strict-Transport-Security (HTTPS 请求时)
- Referrer-Policy

这说明项目已经具备基础安全响应头策略,而不仅仅依赖 Nginx。

## 6.3 XSS 与输入边界

- 留言、标题等纯文本字段走 `cleanText()`。
- 博客、动态等富文本走 `cleanHtml()`。

## 6.4 上传安全

MediaServiceImpl.java:224-283 对上传做了多层校验:

- 文件是否为空
- 文件大小限制
- 扩展名白名单
- MIME 类型白名单
- 图片实际内容校验

## 6.5 来源识别与 HTTPS 判断

backend/src/main/java/com/dyx/blog/common/util/ClientIpUtil.java:12-39

说明项目在来源识别上考虑了反向代理场景:

- 本地或受信代理来源时才尝试读取 X-Forwarded-For
- 对 `::1`、`::ffff`: 等地址格式做归一化
- 通过 X-Forwarded-Proto 辅助判断 HTTPS

这比简单记录 `request.getRemoteAddr()` 更贴近真实部署链路。

## 6.6 路径穿越与对象存储安全

- 本地存储使用 `resolvedPath.startsWith(uploadDirectory)` 防止目录逃逸。
- OSS 凭证不写死在业务配置中,而是从环境变量读取 (`OssMediaStorage.java:176-185`)。

## 6.7 当前未发现的安全/稳定性能力

以下能力在当前仓库中未明确发现:

- OAuth2
- 接口幂等组件
- 熔断/限流框架 (除登录限速外)
- Prometheus / Grafana

- Sentry / ELK
- 分布式缓存

文档中应标注为“当前未发现/未配置”。

## 7. 测试与质量保障

### 5.1 请求生命周期

典型后台请求流程：

1. Nginx 反向代理到 /api/dyx-manager/\*\*
1. SecurityHeaderFilter 写入安全头
1. JwtAuthInterceptor 校验 token 并写入 UserContext
1. Controller 调用 Service
1. Service 调用 Mapper / JdbcTemplate
1. 出错时由 GlobalExceptionHandler 统一转换响应
1. 请求结束后 UserContext.clear() 清理线程上下文

### 5.2 缓存边界

- 开启点：BlogApplication.java:11-13
- 读取缓存：如 SiteServiceImpl.java:95-107, 160-174, 199-287
- 驱逐缓存：如 AdminServiceImpl.java:167-168, 232-233, 275-276, 297-320

缓存明显偏向“公开站点读取多、后台修改少”的博客业务模型。

### 5.3 事务边界

当前已明确看到 @Transactional 用于批量删除访问日志 (AdminServiceImpl.java:132-142) 。  
说明项目没有滥用事务，而是针对多步写操作按需开启。

### 5.4 异常处理流程

GlobalExceptionHandler.java:30-128 将异常分为：

- BusinessException
- 参数校验异常
- 请求方法异常
- 通用兜底异常

返回统一 Result.failure(...) 结构，便于前端 Axios 拦截器统一处理。

## 6. 安全与性能

### 6.1 鉴权与密码安全

- JWT 签发与校验：JwtUtil.java:32-55
- BCrypt 密码校验与升级：AuthServiceImpl.java:86-105
- 登录限速：LoginAttemptLimiter.java:15-76

## 6.2 内容安全

SecurityHeaderFilter.java:31-46 统一设置:

- X-Frame-Options
- X-Content-Type-Options
- X-XSS-Protection
- Content-Security-Policy
- Strict-Transport-Security (HTTPS 请求时)
- Referrer-Policy

这说明项目已经具备基础安全响应头策略，而不仅仅依赖 Nginx。

## 6.3 XSS 与输入边界

- 留言、标题等纯文本字段走 `cleanText()`。
- 博客、动态等富文本走 `cleanHtml()`。

## 6.4 上传安全

MediaServiceImpl.java:224-283 对上传做了多层校验:

- 文件是否为空
- 文件大小限制
- 扩展名白名单
- MIME 类型白名单
- 图片实际内容校验

## 6.5 路径穿越与对象存储安全

- 本地存储使用 `resolvedPath.startsWith(uploadDirectory)` 防止目录逃逸。
- OSS 凭证不写死在业务配置中，而是从环境变量读取 (`OssMediaStorage.java:176-185`) 。

## 6.6 当前未发现的安全/稳定性能力

以下能力在当前仓库中未明确发现:

- OAuth2
- 接口幂等组件
- 熔断/限流框架 (除登录限速外)
- Prometheus / Grafana
- Sentry / ELK
- 分布式缓存

文档中应标注为“当前未发现/未配置”。

# 7. 测试与质量保障

---

## 7.1 测试现状

当前仓库未发现 `backend/src/test/` 或其他自动化测试目录，因此不能把后端测试写成已覆盖。

## 7.2 自动化流水线

当前仓库未发现：

- GitHub Actions
- GitLab CI
- Jenkinsfile

## 7.3 代码规范

虽可从代码风格上看出项目遵循 Java 分层与注释规范，但未发现 Checkstyle / SpotBugs / PMD / Sonar 配置文件，因此静态扫描结果不可虚构。

# 8. 部署与运维

## 8.1 环境配置

主要配置文件：

- backend/src/main/resources/application.yml:1-56
- backend/src/main/resources/application-prod.yml:1-63

其中生产环境通过环境变量注入：

- MYSQL\_HOST
- MYSQL\_PORT
- MYSQL\_USER
- MYSQL\_PWD
- JWT\_SECRET
- ENCRYPT\_KEY

## 8.2 打包与运行

docs/DEPLOY.md:38-61 给出的标准流程为：

```
bash
mvn clean package -DskipTests
nohup java -jar dyx-blog-1.0.0.jar --spring.profiles.active=prod > logs/stdout.log 2>&1 &
```

## 8.3 反向代理

docs/nginx.conf:18-52 将后台 API 与媒体资源代理到 localhost:8080。

## 8.4 存储模式

- 默认本地上传目录：uploads/
- 可选 OSS：系统配置 + 服务端环境变量双重配合

## 8.5 当前未发现的运维能力

- Dockerfile / docker-compose
- K8s ConfigMap
- Nacos / Consul
- 监控告警栈

## 9. 现状结论与补充建议

### 9.1 已具备能力

- Spring Boot + MyBatis-Plus 分层后端
- JWT 后台鉴权
- 登录限速与 BCrypt 密码升级
- 统一异常处理
- 公开数据缓存
- 后台 CRUD 与缓存驱逐
- 媒体上传校验与本地/OSS 双存储
- 安全响应头与 XSS 清洗
- Nginx + 环境变量驱动部署

### 9.2 当前未发现或未实现能力

- 自动化测试体系
- CI/CD
- 容器化部署
- 配置中心
- 监控告警平台
- 文档 lint / 静态扫描工具链

### 9.3 读者复现建议

若读者要快速建立对后端的理解，建议按以下顺序阅读：

1. backend/pom.xml
1. backend/src/main/resources/application.yml
1. backend/src/main/java/com/dyx/blog/BlogApplication.java
1. backend/src/main/java/com/dyx/blog/config/WebMvcConfig.java
1. backend/src/main/java/com/dyx/blog/common/filter/SecurityHeaderFilter.java
1. backend/src/main/java/com/dyx/blog/common/interceptor/JwtAuthInterceptor.java
1. backend/src/main/java/com/dyx/blog/controller/auth/AuthController.java
1. backend/src/main/java/com/dyx/blog/service/impl/AuthServiceImpl.java
1. backend/src/main/java/com/dyx/blog/controller/publics/SiteController.java
1. backend/src/main/java/com/dyx/blog/service/impl/SiteServiceImpl.java
1. backend/src/main/java/com/dyx/blog/controller/admin/AdminController.java
1. backend/src/main/java/com/dyx/blog/service/impl/AdminServiceImpl.java
1. backend/src/main/java/com/dyx/blog/service/impl/MediaServiceImpl.java